INDEX

S No	ANITS Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signature
1	1/7	Introduction to HTML and CSS	CO1	10	
2	8/7	Static webpage using HTML and CSS	CO1	10	
3	15/7	HTML code using audio & video tags	CO1	10	
4	22/7	program using Java Script	CO1	10	
5	29/7	Develop dynamic webpage	CO1	10	
6	5/8	Implement Modules in nodejs	CO2	10	
7	12/8	Mini application using Express & nodejs	CO2	10	
8	19/8	Code tantra (static web page)	CO1	10	
9	2/9	Code tantra (portfolio)	CO1	10	
10	9/9	Code tantra (simple calculator)	CO1	10	
11	16/9	Functional & class component	CO2	10	
12	23/9	HTTP's using JSON Server	CO2	10	
13	30/9	Mongo DB	CO3	10	

Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222. Website: www.anits.edu.in, email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026 - 2027	**Semester:**	1st
Name of the Student:	S. SRI VAMSI	**Roll Number:**	AP24126552179
Name of the Lab:	FS LAB	**Timings:**	9:40 – 12:10

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (10 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4 Marks)	Results (5 Marks)	Viva-Voce (2Marks)			
1/7	01	4	4	5	2	15		
8/7	02	4	4	5	1	14		
15/7	03	4	4	5	2	15		
22/7	04	4	4	5	1	14		
29/7	05	4	4	5	2	15		
5/8	06	4	4	5	1	14		
12/8	07	4	4	5	1	14		
19/8	08	4	4	5	2	15		
2/9	09	3	4	5	2	14		
9/9	10	3	4	5	2	14		

Name of the Faculty: Prasanna Priya

Faculty Signature:

Introduction to HTML and CSS

* HTML (HyperText Markup language) is the standard markup language used to create and structure content on web pages. It defines the basic structure of a webpage using elements and tags such as headings, paragraphs links images, lists tables forms and other multimedia content HTML documents are interpreted by web browsers to display the content in a structured and meaningful manner HTML provides the structure and semantic organization of a webpage

* CSS (cascading style sheets) is a stylesheet language used to control the presentation and visual appearance of HTML documents. It is used to define properties such as colors fonts, backgrounds spacing, borders alignment, dimensions and layouts. CSS also supports responsive design allowing webpages to adapt to different screen sizes and devices CSS can be applied using inline internal or external stylesheets

* HTML and CSS work together to develop webpages where HTML provides the structure and content while CSS provides the styling layout visual presentation the combination of both forms the foundation of modern web development and enables the creation of organized attractive and user-friendly web interfaces.

Difference between MERN Stack and Full Stack

* MERN Stack is a Javascript based technology stack used for developing modern web applications. MERN stands for MongoDB, Express.js, React.js Node.js MongoDB is used for database management Express.js handles the backend framework and API's React.js is used to develop frontend user interface and Node.js provides the server-side runtime environment.

* Full Stack Development refers to the development of both the frontend and backend of a web application along with databases integration A full stack developer works with technologies used for designing the user interface implementing serverside logic managing databases.

HTML Components:

1. HTML Tags - Define the structure and elements of a webpage
2. Headings - used to display titles and headings using <h1> to <h6>
3. Paragraphs - used to display text using the <p> tag
4. Links - used to navigate between webpages using <a> tag
5. Images - used to display images using <image> tag
6. Lists - used to create ordered and unordered lists using <ol>
7. Tables - used to organize data in rows and columns
8. Forms - used to collect user input using elements such as text boxes buttons and check boxes
9. Div and span - used to group and organize webpage content
10. Semantic Elements - Elements such as <header> <nav> <selection> <article> <footer> provide meaningful webpage structure

CSS Components:

1. Selectors – Select HTML elements to apply styles
2. Properties – Define the visual characteristic of elements
3. Values – specify the setting of a css property
4. Colors – Define text, background, border colors
5. Fonts – Control font family, size style and weight
6. Backgrounds – Set background colors and images
7. Borders – Define the border style width and radius
8. Margin – Controls the space outside an element
9. padding – Controls the space inside element
10. Layout – Controls the arrangement of elements using Flexbox Grid and positioning.

HTML VS HTML5

HTML	HTML5
Older version of HTML	Latest major version of HTML
Limited Multimedia support	Supports audio and video
uses external plugins for multimedia	No external plugs needed
Has fewer semantic tags	provides semantic tags
No <canvas> element	Support <canvas>
Limited web Storage	supports local storage
Complex DOCTYPE declaration	Simple <!DOCTYPE HTML>

CSS VS CSS3

CSS	CSS3
Earlier version of CSS	Advanced version of CSS
Basic Styling features	provides advanced styling features
Limited animations	supports animations and transitions
No flexible layout systems	Supports flex box and Grid
Limited visual effects	supports shadows, gradients, effects
No media queries	supports media queries
uses a single specification	Divided into different modules.

Design a Static webpage using HTML and CSS

Pomodoro Timer using HTML, CSS

```
<!DOCTYPE html>
<html>
<head>
    <title Pomodoro Timer </title>
    <Style>
        body {
         font-family : Arial;
         background : # f5f5f5;
         text_align : center;
          padding: 50px;
        }
        .timer {
            width : 300px ;
            margin : auto;
            padding : 30px;
            background : white;
            border-radius : 15px;
            box-shadow : 0 0 10px gray;
        }
        h1 {
         color : #e74c3c ;
        }
```

```
.time {
  font-size : 50px;
  margin : 20px;
  }
button {
    padding: 10px 20px;
    margin : 5px;
    border : none;
    border-radius : 5px;
    background : # e74c3c;
    color : white;
    }
    </style>
</head>
<body>
  <div class= "Timer">
    <h1> Pomodoro Timer </h1>
    <div class = "time " > 25:00 </div>
    <button> start </button>
    <button> Reset </button>

  </div>

</body>
</html>
```

O/p:



(i) Design an HTML code using audio tags

code:

```
<!DOCTYPE html>
<html>
<head>
    <title> Audio Example </title>
</head>
<body>
    <h2> My favorite song </h2>
    <audio controls>
        <source src = "song.mp3" type =
        "audio/mpeg">
        your browser doesnot support the
        audiotag
    </audio>

</body>
</html>
```

O/p: Displays an audio player with play, pause and volume controls

Steps:

Create a new HTML file

Add the basic HTML structure

Add heading using <h2> tag

Insert the <audio> tag with controls attribute

Specify the audio file using <source> tag

Save and open the HTML file in a web browser

Play the audio using displayed controls.



(ii) Design a HTML code using video tags.

Code:

```
<! DOCTYPE html>
 <html>
 <head>
     <title> Video Example </title>
 </head>
 <body>
     <h2> My Video </h2>
     <video width = "400" controls >
      <source src = "video.mp4" type =
       " video/mp4 ">
      </video>
 </body>
</html>
```

Displays a video player with play, pause volume and Full screen controls

Create a new HTML file
Add a basic HTML structure
Add a heading using the <h2> tag
Insert the <video> tag with controls attribute
Specify the video file using <source> tag
Save the file and open it in web browser
play the video using displayed controls

Write a program using Javascript Arrays, functions

Code :

```
let numbers = [12, 45, 8, 23, 56, 19, 34, 67, 90, 11];

function displayArray(arr){
    console.log("Array Elements:");
    for(let i=0; i<arr.length; i++){
        console.log(arr[i]);
    }
}

function findSum(arr){
    let sum=0;
    for(let i=0; i<arr.length; i++){
        sum += arr[i];
    }
    return sum;
}
```

Develop dynamic webpage using javascript

program:

```
<! DOCTYPE html>
<html>
<head>
  <title> Dynamic webpage </title>
  <style>
     body {
     text_align : center;
     font_family : Arial;
     margin_top : 100px;
     }
   button {
     padding : 10px 20px;
     font_size : 16 px;
     cursor : pointer;
     }
  </style>
</head>
<body>
 <h1 id = "heading" > Welcome to my
     webpage </h1>
 <p id = "message" > Click the button to
 change this webpage </p>
```

```
<button onclick = " changepage() "> ClickMe </
    button>

<script >
    function changepage(){
      document.getElementById("heading")
      innerHTML = " welcome to javascript "
    document.getElement ById ("heading")
      innerHTML = " The webpage has been
      changed dynamically "
    document. body.style.backgroundcolor = "light"
      }
    </script>
    </body>
    </html>
```

Output:

Welcome to my webpage

Click the button to change this webpage

ClickMe

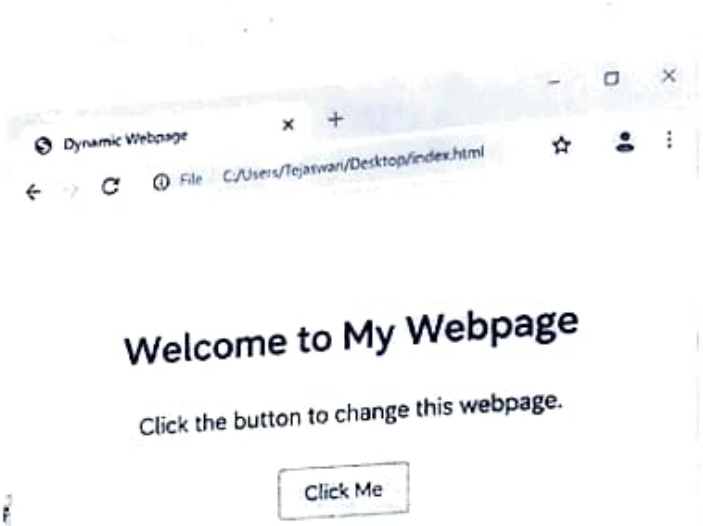
When the user clicks "ClickMe" the javascript function changepage() is called then the heading dynamically changes and background also changes

After clicking the button

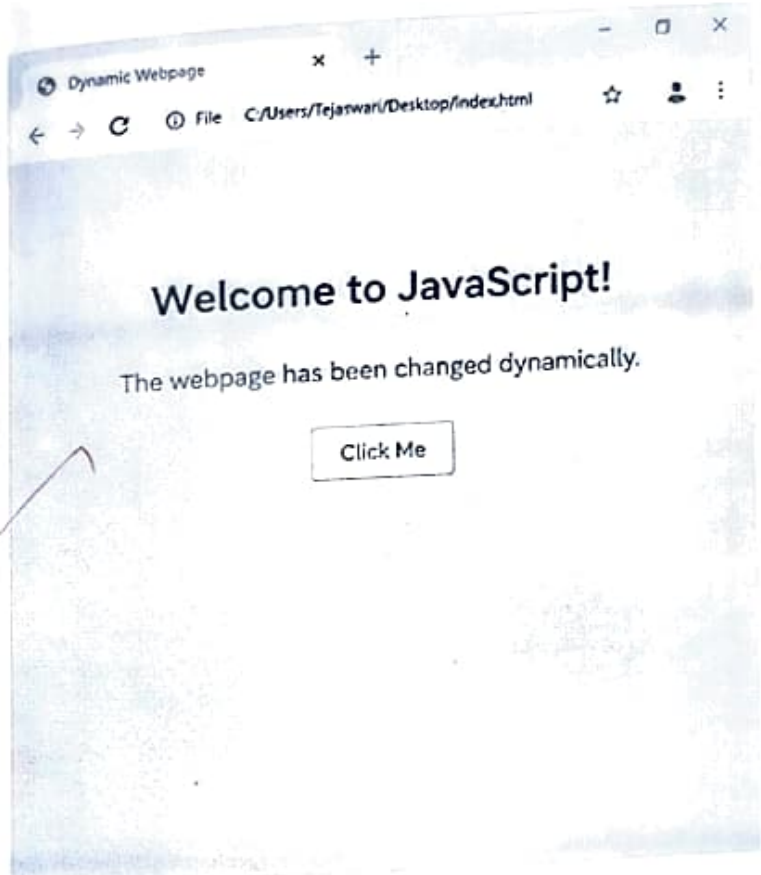
Welcome to Javascript

The webpage has been changed dynamically

ClickMe



Before clicking the button



After clicking the button

15

5

Implement Modules in NodeJS

In Node.js you can implement modules using distinct systems: CommonJS(JS) which is traditional default, and ES modules(ESM) the modern javascript standard

1 The common Js Method (default)

CommonJS uses module exports to share code and require() to import it This system loads modules synchornously

a create the module (math.js)

```
const add = (a,b) => a+b;
Const a Subtract = (a,b) => a-b;
module exports = {
        add,
        Subtract,
      };
```

b use the module (app.js)

```
Const math = require('./math');
Console.log(math.add(10,5));
Console.log(math.Subtract(10,5));
```

Output :

15

5

2 The Es modules Method(modern Standard):

Es modules use export to share code and import to use it. To make node.js recognize Es modules, you must either use the .mjs file extension or add "type" "module" to your package .json file

a create the module (calculator.js)

```
export const multiply = (a,b) => a*b
export const divide = (a,b) => a/b
const calculatorName = "supercalc";
export default calculatorName;
```

b use the module (main.js)

```
import calculatorName { multiply divide }
                from './calculator.js';
    console.log(calculatorName);
    console.log( multiply (4,2));
    console.log( divide (4,2));
```

Output:

```
Supercalc
8
2
```

3 using Built-in core modules

Node.js also comes equipped with dozens of builtin core modules (like fs for filesystem or path for file routing that don't need installation

We will load them by module names

```
// commonJs style
const fs = require ('fs');

// ES modules style
import fs from 'fs';
```

Here are some of the most commonly used ones;

1 fs = file system operations

2 http = HTTP server and client

3 path = file path utilizes

4 OS - operating system utilities

5 events - Event handling

6 util - utility functions

7 stream - Stream handling

8 crypto - cryptographic functions

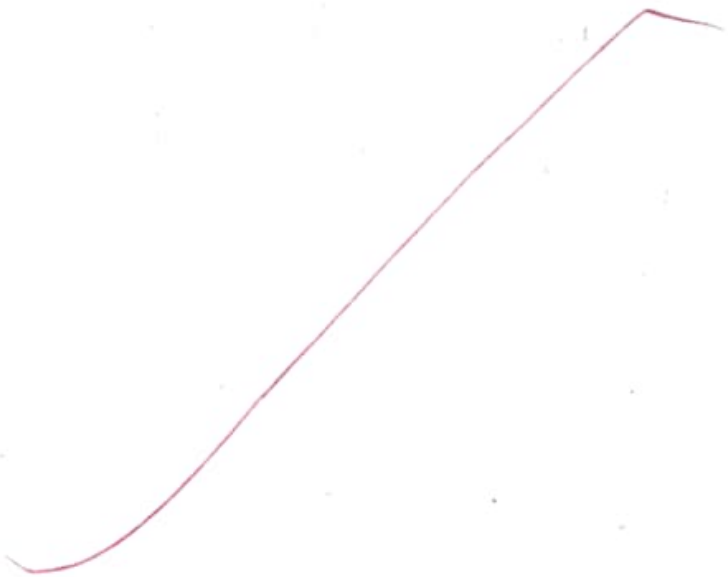
9 URL - URL parsing

10 querystring - URL query string handling

Eg: Simple HTTP Server

```
http.createServer((req, res) => {
    res.writeHead(200, {'Content-Type':
                        'text/html'});
    res.End("Hello world!");
}).listen(8080);
```

O/p: Hello World!



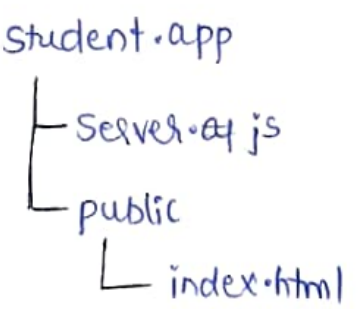
Develop mini application using express and NodeJS

Simple mini application using nodeJS + ExpressJS

Student Management System

1 Add a student
2 View all students
3 delete a student
4 uses Express js routes
5 Simple HTML interface

project Structure



Server.js

```
const express = require("express");
Const app = express();
const PORT = 3000;
let students = [];
app.use(express.json());
app.use(express.static("public"));
```

```
app.get("/students", (req, res) => {
    res.json(students);
});
// Add
app.post("/students", (req, res) => {
    const student = {
        id: students.length + 1,
        name: req.body.name,
        course: req.body.course,
    };
    students.push(student);
    res.json({ message: "student added
                successfully",
        student: student });
});
// Delete
app.delete("/students/:id", (req, res) => {
    const id = parseInt(req.params.id);
    students = students.filter(student =>
        student.id !== id);
    res.json({
        message: "student deleted successfully" });
});
```

```
// start server
app.listen(PORT, () => {
  console.log('server running at http://localhost
      ${PORT}');
  });
public/ index.html
<!DOCTYPE html>
<html>
<head>
  <title> Student Management </title>
</head>
<body>
  <h1> Student Management system </h1>
<input type = "text" id = "name"
 placeholder = "Enter Student name" >
<input type = "text" id = "course"
 placeholder = "Enter Course" >
<button onclick = "addStudent()" > Add Student
                                  </button>
<h2> Student List </h2>
<ul id = "StudentList" ></ul>
<script>
 function addStudent(){
  let name = document.getElementById("name").
                value;
```

```
let course = document.getElementById("course").
                value;
fetch("/students", {
    method: "POST",
    headers: {
        "Content-Type": "application/json"},
    body: JSON.stringfy({
        name:name,
        course: course,
    })
})
.then(response => response.json())
.then(data => {
    alert(data.message());
document.getElementById("name").value = "";
document.getElementById("course").value = "";
    displayStudents()
});
}
```

```
function displayStudents(){
  fetch("/students").then(response => response.
                                    json()))

  .then(students => {
  let list = document.getElementById("StudentList");
  list.innerHTML = "";
Students = forEach(student => { let li =
document.createElement("li");
    li.innerHTML = student.id + "_" +
                   Student name + "_" +
                   Student course + "<button>
Onclick = 'deleteStudent(" + Student.id + ")'>
  Delete </button>";
list.appendchild(li);
          });
        });
      }
    function  deleteStudent(id){
      fetch("/students/" + id { method
    "DELETE "})
    .then(response => response.json())
    .then(data => {
          alert(data.message);
          displayStudents();
        });
```

```
    }
    displayStudents();
  </script>
</body>
</html>
```

Output :

Student Management System

Enter student name

Enter course

Add Student

Student List :

1 Vikram CSM delete

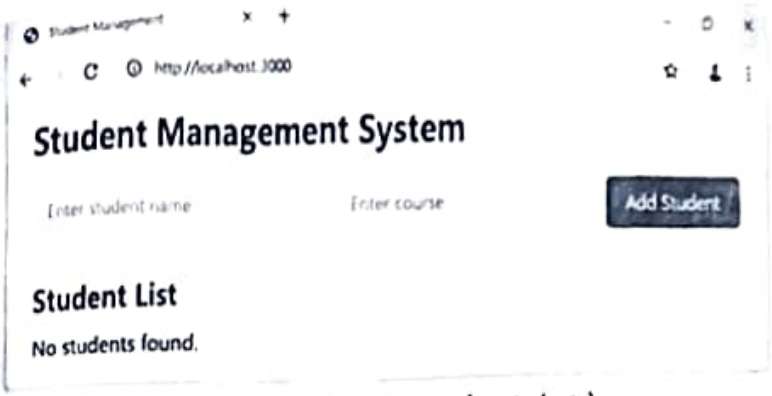
2 Surya ECE delete

The Express.js server is successfully started at : http// localhost.3000

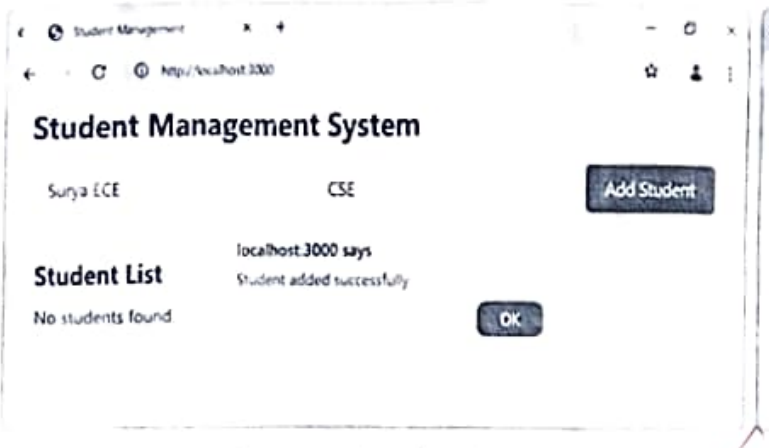
The browser displays the student management system with fields.



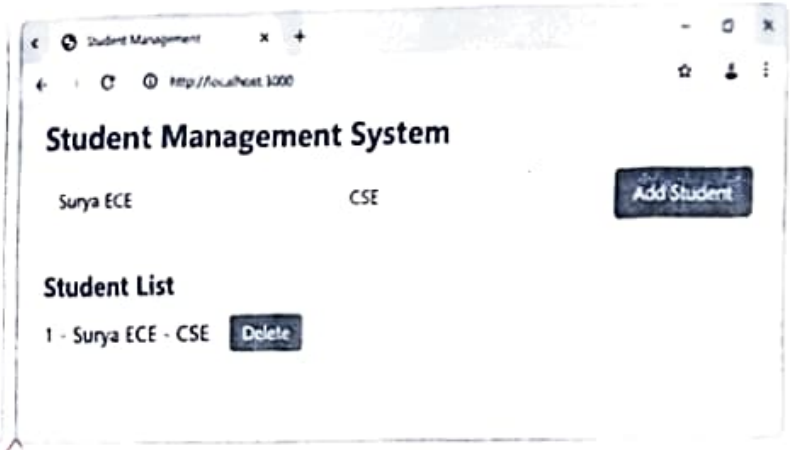
1. Terminal Output (after running `node server.js`)



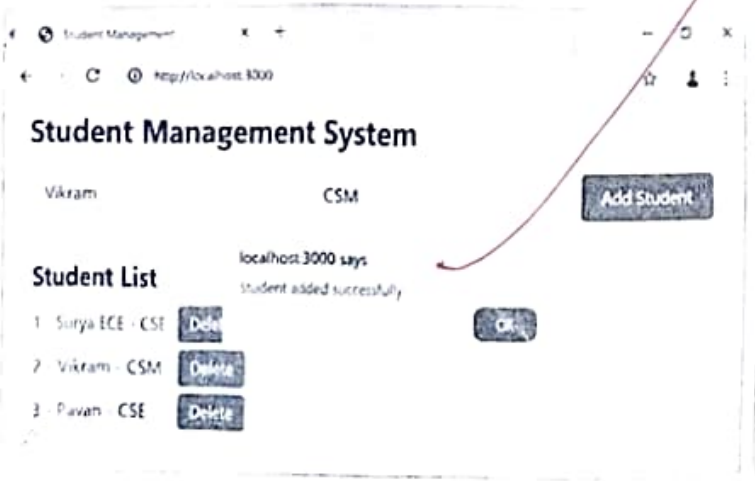
2. Initial Output in Browser (no students)



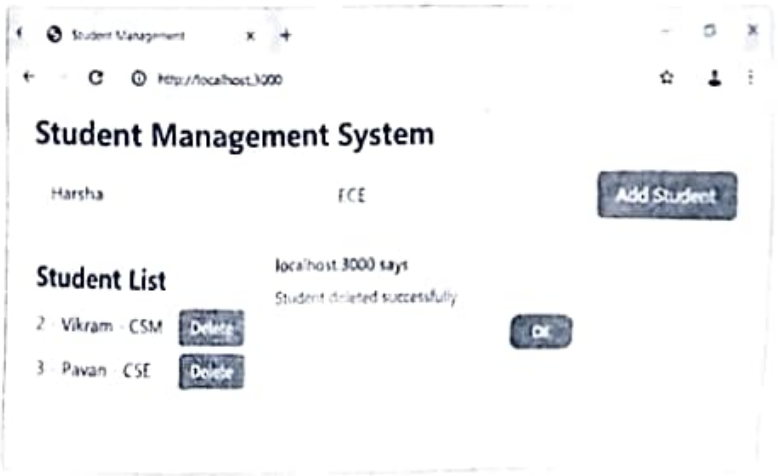
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

Experiment – 07

Create a functional component and class component using react

Aim: To develop a react application using functional components and class components to display student details, manage state handle button clicks and dynamically update data on a web page.

functional component:

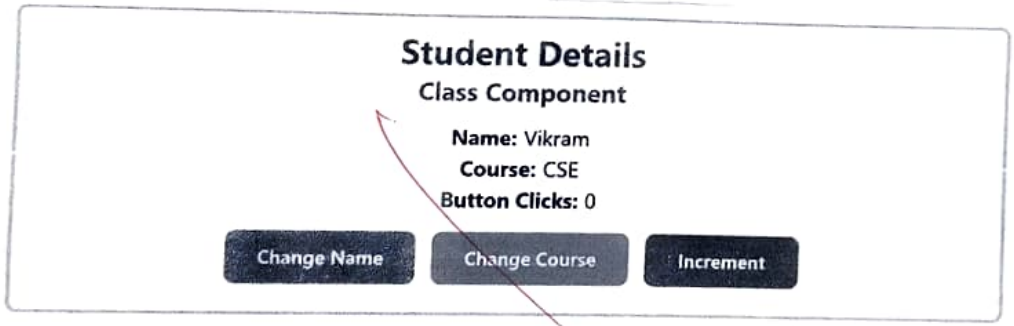
A functional component is a javascript function that returns JSX to display the user interface. It uses React Hooks such as usestate to manage state and update the webpage dynamically

program:

```
import React, {useState} from
 'react';
function studentfunction() {
      const [name, setName] = useState("Vamsi");
     const [course, setcourse] = usestate("csm");
     const [count, setcount] = usestate(0);
     const changeName = () => {
          setName("Sri");
        };
    const changeCourse = () => {
         setcourse("CSE");
        };
    const incrementcount = () => {
         setcount(count+1);
       };
```

```
return (
<div style={{
    border: "2px solid blue",
    padding: "20px",
    margin: "20px",
    borderRadius: "10px",
    textAlign: "center"
}}>
<h1>Student Details</h1>
<h2>Functional Component</h2>
<p><b>Name:</b>{name}</p>
<p><b>course:</b>{course}</p>
<p><b>Button clicks:</b>{count}</p>
<button onclick={changeName}>
    changeName</button>
<button onclick={changeCourse}>change
    course</button>
<button onclick={incrementCount}>Increment
    </button>
</div>
);
```

Output: When the user clicks the buttons, the name course and count change dynamically without reloading the webpage.


Student Details
Class Component
Name: Vikram
Course: CSE
Button Clicks: 0
Change Name
Change Course
Increment

Class Component:

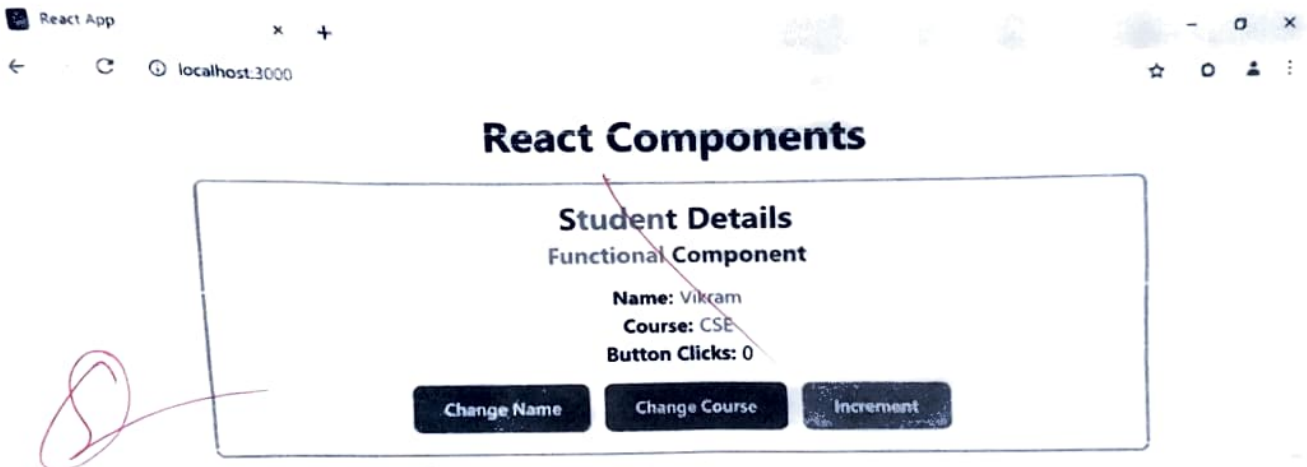
A class component is an ES6 Javascript class that extends React component. It uses the render() method to return JSX and manages state using this.state and this.setState().

program:

```
import React { Component } from
'react';
class Studentclass extends Component {
    constructor (props) {
        super(props);
        this.state = {
            name : "Vikram",
            course : "Computer Science",
            count : 0
        }
    }
changeName = () => {
    this.setState( {
        name : "vamsi"
        });
    };
changecourse = () => {
    this.setState({
        course : "CSM",
    });
};
```

```
incrementCount= ( )=> {
    this.setState((prevState )=> ({
        count: prevState.count+1
        }));
    };
render(){
  return(
    <div style={{
        border: "2px solid green",
        padding: "20px",
        margin: "20px",
        borderRadius: "10px",
        textAlign: "center",
        }}>
      <h1> Student Details</h1>
      <h2> class component</h2>
    <p> <b>Name:</b> {this.state.name}</p>
   <p> <b> Course:</b> {this.state.course }</p>
   <p><b>Buttonclicks </b>{this.state.count}</p>
<button onclick= {this.changeName}>
        changeName </button>
<button onclick ={this.change course}>
        ChangeCourse </button>
<button onclick={this.incrementCount} >
        Increment </button>
    </div>
```

Output:


React App
localhost:3000
React Components
Student Details
Functional Component
Name: Vikram
Course: CSE
Button Clicks: 0
Change Name
Change Course
Increment

Experiment - 08

Implement http methods using JSON Server

Aim: To implement HTTP methods such as GET POST PUT PATCH and DELETE using JSON Server to perform CRUD (Create, Read, update and delete) operations on student reacts

Theory:

JSON server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operations without developing a backend server from scratch

HTTP methods are used to communicate between a client and a server

GET - Retrives data from the server

POST - Adds new data to the server

PUT - Replaces an existing record

PATCH - updates selected fields of a record

DELETE - Removes a record from the server

a Create a JSON Server

Step 1 : Initialize the project

open the vscode terminal and execute the following commands

```
mkdir JSONServer
cd JSONServer
npm init -y
npm install json server
```

Step2: Create db.json

create a file named db.json and add the following student data

```
{
"students": [
    {
    "id": "1",
    "name": "vikram",
    "course": "CSM",
    "marks": 90
    },
    {
    "id": "2"
    "name": "vamsi",
    "course": "CSE",
    "marks": 90
    },
    {
    "id": "3"
    "name": "swathi"
    "course": "IT"
    "marks": 97
    }
```

Step3 Start the JSON Server

For the current JSON Server version, execute

npx json-server db.join

The server start at

http://Localhost:3000

The student records can be accessed using

http://http localhost: 3000/students

2 Implementation of HTTP Methods

A GET Method: The GET Method is used to retrieve student records from JSON server

Code:

```
fetch('http://Localhost:3000/students').
.then(response => response.json())
.then(data=>{
        console.log(data);
    });
.catch(error =>{
    console.log(error);
    });
```

output:

1. GET All Students (http://localhost:3000/students)



B POST Method: The POST method is used to insert a new student record into the server

code: fetch('http://localhost:3000/students',{

method: 'POST',

headers {

'Content-Type': 'application/json'

},

body: JSON.stringify({

name: 'Rahul',

Course: 'CSE'

marks: 92,

})

})

.then(response => response.json())

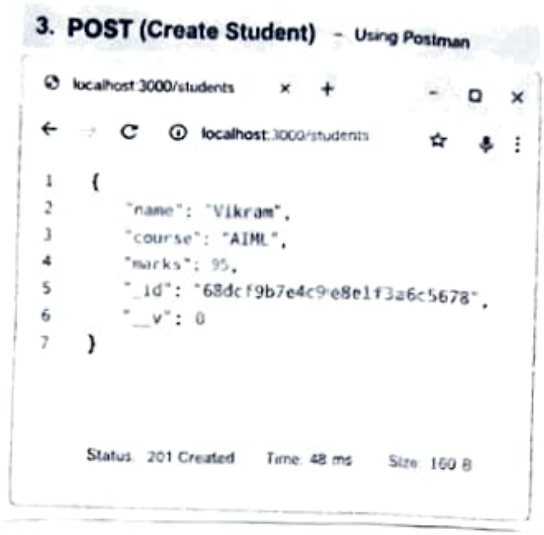
.then(data => {

console.log(data);

})

```
• catch(error => {
    console.log(error);
});
```

output:



C PUT Method: The PUT method is used to replace an existing student record with updated data

Code:

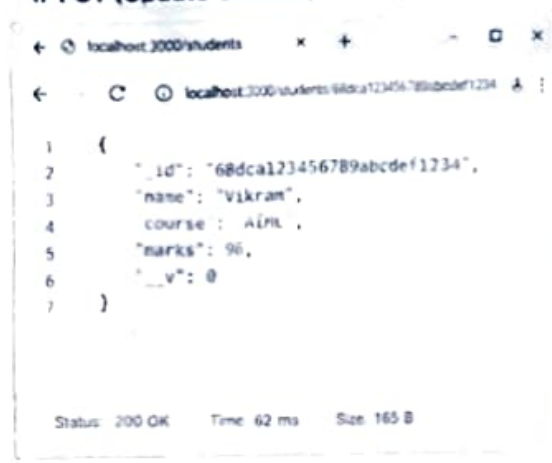
```
fetch('http://localhost:3000/students/1', {
    method: 'PUT'
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
       name: 'Vikram',
       course: 'AIML',
       marks: 97
    })
})
```

```
• then(response => response.json())
• then(data => {
```

```
})
.catch(error=>{
    console.log(error);
});
```

output:

4. PUT (Update Student) - Using Postman



D PATCH method: The PATCH method is used to update only selected fields of an existing Student record

code

```
fetch('http://localhost:3000/students/2',{
    method: 'PATCH',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.Stringify({
      marks :96
    })
})
.then(response => response.json())
.then(data => {
    console.log(data);
```

```
.catch(error => {
    console.log(error);
});
```

Output:

E DELETE Method: The DELETE method is used to remove an existing student record from the server

Code:

```
fetch('http://localhost:3000/students/3', {
    method: 'DELETE'
})
.then(response => {
    if (response.ok){
        console.log("Student deleted successfully");
    }
})
.catch(error => {
    console.log(error)
});
```

After executing the POST, PUT, PATCH and DELETE operations in sequence the remaining student will be

Output:



Thus HTTP methods GET POST PUT PATCH and DELETE were successfully implemented using JSON server the CRUD operations were performed on Student records the data was retrieved inserted updated and deleted using HTTP requests.

Experiment - 09

Develop CRUD operations using MongoDB

Aim: To develop a web application using Node.js Express.js and mongoDB to perform CRUD operations (create, read, update, delete) on Student records

Theory: MongoDB is a NOSQL database that stores data in flexible JSON like documents CRUD operations are used to manage data in a database.

Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js It helps define schemas, interact with collections and perform database operations

1 Installation and Project Setup:

```
mkdir MongoDB_CRUD
cd MongoDB_CRUD
npm init -y
npm install express mongoose
```

program : server.js

```
Const express, require('express');
Const mongoose : require('mongoose');
Const app= express();
Const PORT = 3000;
app.use(Express.json());
```

```
mongoose.connect('mongoDB://127.0.0.1:27017/college')
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));

// Create Student
const StudentSchema = new
mongoose.Schema({
        name: {
          type: String
          required: true
        },
        Course: {
          type: String,
          required: true
        },
        masks: {
          type: Number
          required: true
        }
      });
// Create Student Model
const Student =
    mongoose.model('Student', StudentSchema);
```

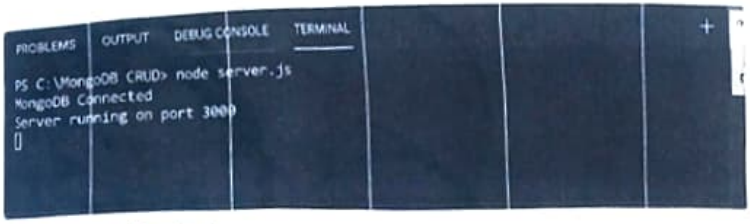
```
// Create : Add
app.post('/students' asyne req, res )=> {
    try {
     const student= new student(req.body );
     const saved student = await student save();

res. Status (201). json (savedstudent);
   } catch(error){
       res.status(400).json({ message
                error. message });
              }
          });
// Read get all students
app.get('/students', asyn( req, res )=> {
     try {
       const students - await
       Student. find();
       res. status (500).json({ message : error:
                                message });
      }
   });
// Read : Get Student by ID
app.get('/students /id' asyn( req, res )=>{
     try {
      const student = await
      student findByJd (req . params id );
```

```
    if(!student){
      return res.status(404).json({
        message: "Student not found"
      });
    }
    res.status(200).json(student)
  }catch(error){
    res.status(400).json({ message:
      error.message });
  }
});

// update
app.put('/students/id', async(req,res)=>{
  try{
    const student = await
    student findById and update(
      req.params.id,
      req.body,
      { new:true, runValidators:true }
    );
  if(!student){
    return res.status(404).json({
      message:" student not found",
    });
  }
```

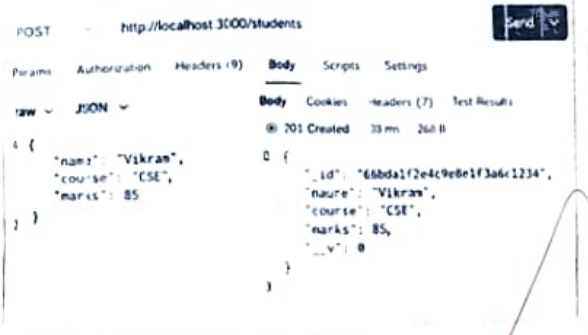
```
    res.status(500).json(student);
  } catch (error) {
    res.status(400).json({ message
        error.message });
  }
});
// Delete
app.delete('/students/:id', async(req, res)=>{
  try {
    const student await Student
        findById s and Delete(
          req.params id
        );
    if(!student){
      return Ne res.status(404).json({
        message "student not found"
      });
    }
    res.status(200).json({
      message: "Student deleted successfully"
    });
  } catch(error) {
    res.status(400).json({message
        error.message });
  }
});
```



1. Terminal Output



2. Chrome Browser (Home Page)



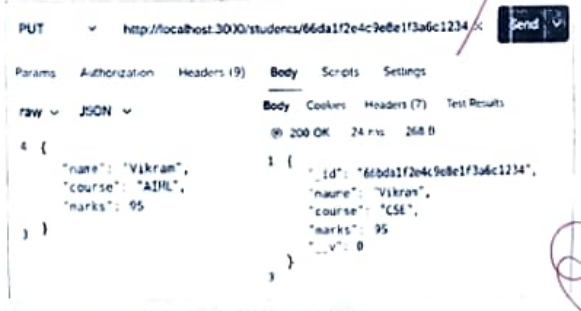
3. Create (POST) – Add Student



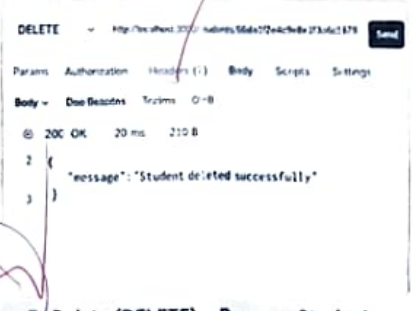
4. Read (GET) – Get All Students



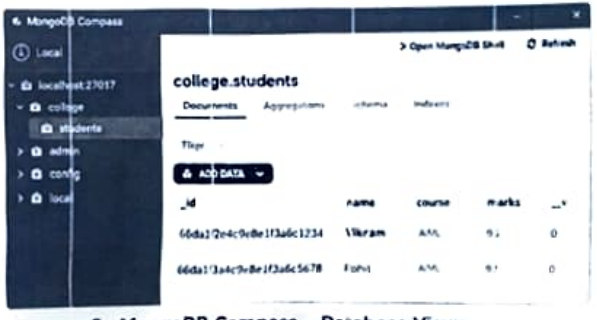
5. Read (GET) – Get Student by ID



6. Update (PUT) – Update Student



7. Delete (DELETE) – Remove Student



8. MongoDB Compass – Database View